

## ЛАБОРАТОРНАЯ РАБОТА 3

Существуют различные технологии анализа данных: написание кода и/или применение low-code инструментов. Low-code инструменты — готовые виджеты и конструкторы позволяют значительно сократить время на создание приложения без потерь в качестве и гибкости готового решения. Удобство low-code инструментов не вызывает сомнений, однако для освоения методов работы с большими данными необходимо обладать навыками программирования.

### Интерпретаторы исполняемого кода

В платформе Loginom есть интерпретаторы кода для двух языков программирования: **Python** и **JavaScript** [1].

Для работы с **Python** потребуется установить интерпретатор языка на компьютер, где инсталлирован Loginom, и провести предварительную настройку. Запускать написанный скрипт можно внутри процесса Loginom или в отдельном процессе (в процессе интерпретатора).

**JavaScript** работает «из коробки» без каких-либо дополнительных настроек, инсталляций и иных действий. Помимо узла JavaScript, этот язык программирования можно использовать в обработчике Калькулятор, позволяющем писать формулы как в стиле Excel, так и на JavaScript.

Также в маркетплейсе Loginom представлена библиотека **Python Kits**, которая позволяет автоматизировать рутинные операции с наборами данных, переносить существующий код на **Python** в платформу, разрабатывать универсальные шаблоны сценариев и внедрять в сценарий модели машинного обучения из библиотек на **Python**.

В справочных материалах Loginom рекомендуется использовать **Python** не для запуска алгоритмов машинного обучения, а для решения простых задач, таких как добавление новых записей в зависимости от значений определенных полей, видоизменения таблицы с использованием циклов, преобразования строк JSON в объект и т.п. [1].

Рассмотрим использование **узла Python** для решения задачи построения и отбора признаков.

**Признак** (feature) – это переменная (столбец в таблице), которая описывает отдельную характеристику объекта. Признаки являются основой для задач машинного обучения в целом: именно на их основании строятся предсказательные модели.

**Построение признаков** (feature engineering) – это процесс, во время которого извлекаются новые переменные из необработанных данных. Рабочие процессы аналитики больших данных зависят от построения и отбора признаков.

### Удаление пропущенных значений

**Отсутствующие значения** –распространенная проблема данных. Пустые значения влияют на производительность моделей машинного обучения.

Самое простое решение проблемы – отбросить строки или весь столбец с пустыми ячейками. Оптимального порога для отбрасывания не существует, но можно задать порог и отбросить строки со столбцами, в которых отсутствуют значения, превышающие этот порог. Значение порога устанавливается с учетом контекста данных.

**Компонент Python** (рис.1) содержит редактор исполняемого кода (рис.2). На примере удаления пропущенных значений рассмотрим алгоритм получения входного набора, его обработку и формирование выходного набора.

Исследование на рис.1 выполнено для набора данных data.csv, содержащего сведения о пациентах и столбец TenYearCHD – прогнозирование сердечных заболеваний. В наборе нет уникального идентификатора прецедентов; с помощью калькулятора был добавлен столбец ID с формулой RowNum()+1. Статистический анализ набора показал значительное количество пропущенных данных.

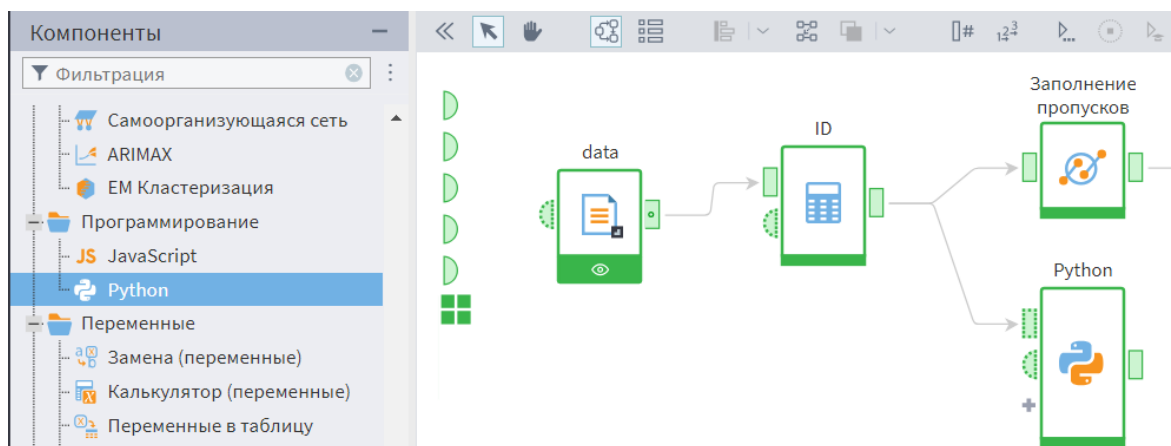


Рис. 1. Использование узла Python для подготовки данных к анализу

Сравним возможности компонентов Loginom и алгоритмы Python для обработки данных как показано на рис.1.

## Лабораторные работы к дисциплине Большие данные

```
##Модуль API для Loginom
import builtin_data
from builtin_data import InputTable, InputTables, InputVariables, OutputTable, DataType, DataKind, UsageType
#Модуль Pandas используется для обработки и анализа данных
#NumPy – низкоуровневая структура данных, поддерживающая многомерные массивы и математические операции с массивами
import numpy as np, pandas as pd
#Модуль, в котором реализован метод to_data_frame для создания pandas.DataFrame по набору данных входного порта
from builtin_pandas_utils import to_data_frame, prepare_compatible_table, fill_table
if InputTable:
    # Создать pd.DataFrame по входному набору
    data = to_data_frame(InputTable)
#Установим порог отсеивания
shutoff = 0.5
# Удаление столбцов с коэффициентом пропущенных значений выше порога
data = data[data.columns[data.isnull().mean() < shutoff]]
# Удаление строк с коэффициентом отсутствия значений выше порога
data = data.loc[data.isnull().mean(axis=1) < shutoff]
# Полученный выходной pd.DataFrame
output_frame = pd.DataFrame(data)
# Если включена опция "Разрешить формировать выходные столбцы из кода", структуру выходного набора можно подготовить по pd.DataFrame
if isinstance(OutputTable, builtin_data.ConfigurableOutputTableClass):
    prepare_compatible_table(OutputTable, output_frame, with_index=False)
fill_table(OutputTable, output_frame, with_index=False)
```

Рис. 2. Редактор исполняемого кода

Для исключения пустых значений визуальными средствами платформы Loginom следует использовать **компонент Заполнение пропусков** (рис.1). Пример настроек функциональности узла показан на рис.3.

### Заполнение пропусков

Исходные данные упорядочены ☐

Допустимый процент пропусков

Random seed

| № | Входные поля                            | Вид данных  | <input type="checkbox"/>            | Метод обработки                |
|---|-----------------------------------------|-------------|-------------------------------------|--------------------------------|
|   | <input type="text" value="Фильтрация"/> |             |                                     |                                |
| 1 | 12 ID                                   | Непрерывный | <input type="checkbox"/>            | Не выбран                      |
| 2 | 12 male                                 | Непрерывный | <input checked="" type="checkbox"/> | Удалять записи                 |
| 3 | 12 age                                  | Непрерывный | <input checked="" type="checkbox"/> | Заменять медианой              |
| 4 | 12 education                            | Непрерывный | <input checked="" type="checkbox"/> | Заменять случайными значениями |
| 5 | 12 currentSmoker                        | Непрерывный | <input checked="" type="checkbox"/> | Заменять средним               |
| 6 | 12 cigsPerDay                           | Непрерывный | <input checked="" type="checkbox"/> | Заменять на 0                  |
| 7 | 12 BPMeds                               | Непрерывный | <input checked="" type="checkbox"/> | Заменять наиболее вероятным    |

Рис. 3. Методы обработки узла Заполнение пропусков

### Заполнение пропущенных значений

Заполнение пропущенных значений — более предпочтительный вариант, чем отбрасывание, так как сохраняется размер данных. При этом важно подобрать механизм заполнения (рис.4), например, можно использовать медианы столбцов. Поскольку средние значения столбцов чувствительны к значениям выбросов, медианы в этом отношении будут более устойчивыми.

```

13 #Заполнение пропущенных значений столбцов максимальными значениями
14 data['education'] = data['education'].fillna(data['education'].max()).astype(int)
15 #Заполнение пропущенных значений столбцов минимальными значениями
16 data['cigsPerDay'] = data['cigsPerDay'].fillna(data['cigsPerDay'].min()).astype(int)
17 data['BPMeds'] = data['BPMeds'].fillna(data['BPMeds'].min()).astype(int)
18 data['totChol'] = data['totChol'].fillna(data['totChol'].min()).astype(int)
19 #Заполнение пропусков конкретными значениями
20 data['BMI']=data['BMI'].fillna(20).astype(int)
21 #Заполнение пропусков медианными значениями
22 #Найти медианное значение конкретного столбца
23 median_value1 = data['heartRate'].median()
24 median_value2 = data['glucose'].median()
25 #Заполнить все пропущенные значения медианным значением
26 data['heartRate'].fillna(median_value1, inplace=True)
27 data['glucose'].fillna(median_value2, inplace=True)
28
29 #Заполнение пропусков медианными значениями всех столбцов выполняет метод fillna()
30 data.fillna(data.median(), inplace=True)

```

Рис.4.4. Заполнение пропущенных значений

Замена отсутствующих значений на максимальное значение в столбце будет хорошим вариантом для работы только в случае, когда мы разбираемся с категориальными признаками.

### Обнаружение выбросов

Выявление и устранение выбросов является важной частью управления данными. Экстремальные значения влияют на результат анализа, поэтому важно знать и использовать способы снижения их негативного воздействия.

**Выбросом** (outlier) считается значение в данных, которое находится далеко за пределами других наблюдений.

Один из способов обнаружения выбросов – рассчитать стандартное отклонение. Если значение отклоняется больше, чем на  $x$  стандартное отклонение, его можно принять, как выброс. Наиболее часто используемое значение для  $x$  – в пределах [2, 4]. Удаление строк с выбросами показано на рис.5.

```

13 # Избавляемся от выбросов с помощью стандартного отклонения
14 x=3
15 upper_lim = data['cigsPerDay'].mean()+data['cigsPerDay'].std()*x
16 lower_lim = data['cigsPerDay'].mean()-data['cigsPerDay'].std()*x
17 data = data[(data['cigsPerDay'] < upper_lim) & (data['glucose'] > lower_lim)]

```

Рис.5. Удаление выбросов с помощью стандартного отклонения

Другой математический метод обнаружения выбросов – использование процентилей (рис.6). Устанавливается определенный процент значения сверху или снизу за выброс.

```
19 # Избавляемся от выбросов при помощи процентилей
20 upper_lim = data['glucose'].quantile(.95)
21 lower_lim = data['glucose'].quantile(.05)
22 data = data[(data['glucose'] < upper_lim) & (data['glucose'] > lower_lim)]
```

*Рис.6. Удаление строк с выбросами с использованием процентилей*

Альтернативный способ обработки выбросов – ограничить их, а не отбрасывать. Сохранение размера данных может быть лучшим решением для окончательной производительности модели (рис.7).

```
24 # Ограничение выбросов без потери объема набора
25 upper_lim = data['glucose'].quantile(.95)
26 lower_lim = data['glucose'].quantile(.05)
27 data.loc[(data['glucose'] > upper_lim), 'glucose'] = upper_lim
28 data.loc[(data['glucose'] < lower_lim), 'glucose'] = lower_lim
```

*Рис.7. Ограничение строк с выбросами без потери объема данных*

В Logiном компонент **Редактирование выбросов** предназначен для автоматической корректировки выбросов и экстремальных значений в наборах данных. Для каждого поля исходного набора данных критерии определения выбросов и экстремальных значений задаются пользователем с помощью указания допустимого стандартного отклонения или интерквартильного размаха. (рис.8).

## Редактирование выбросов

Исходные данные упорядочены

☐

| Входные поля                        |               | Вид данных  |
|-------------------------------------|---------------|-------------|
| Фильтрация                          |               |             |
| <input type="checkbox"/>            | 12 prevalence | Непрерывный |
| <input type="checkbox"/>            | 12 diabetes   | Непрерывный |
| <input type="checkbox"/>            | 12 totChol    | Непрерывный |
| <input type="checkbox"/>            | 9.0 sysBP     | Непрерывный |
| <input type="checkbox"/>            | 9.0 diaBP     | Непрерывный |
| <input type="checkbox"/>            | 9.0 BMI       | Непрерывный |
| <input type="checkbox"/>            | 12 heartRate  | Непрерывный |
| <input checked="" type="checkbox"/> | 12 glucose    | Дискретный  |
| <input type="checkbox"/>            | 12 TenYearCHD | Непрерывный |

Определение выбросов и экстремальных значений

☐ Оставить без изменения
 ☐ Интерквартильная ширина

Удалять записи

Для экстремального

Метод обработки экстремальных значений

Заменять медианой

Заменять наиболее вероятным

Заменять заданным значением

Ограничивать

Заменять медианой

Рис. 8. Методы обработки узла Редактирование выбросов

## Масштабирование признаков

В большинстве случаев числовые характеристики набора данных не имеют определенного диапазона и отличаются друг от друга.

Например, столбцы возраста и месячной зарплаты будут иметь совершенно разный диапазон.

Как сравнить эти два столбца, если это необходимо в задаче? **Масштабирование** решает эту проблему, так как после данной операции элементы становятся идентичными по диапазону. Существует два распространенных способа масштабирования: нормализация и стандартизация.

Выбор между этими методами зависит от конкретной задачи. Если важно, чтобы все значения находились в определенном диапазоне, например, для использования в алгоритмах, чувствительных к масштабу, то следует использовать **нормализацию**.

Если же важно, чтобы данные имели определенное распределение, например, для использования в алгоритмах, которые предполагают нормальное распределение, то следует использовать **стандартизацию**.

## Нормализация

**Нормализация** преобразует данные так, чтобы все значения находились в диапазоне от 0 до 1.

В Python нормализация данных реализуется несколькими методами библиотеки Pandas. На рис.9 приведен пример использования функции масштабирования min-max.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
data=pd.read_excel(r"Нормализация.xlsx")
for column in data.columns:
    data[column] = (data[column] - data[column].min()) / (data[column].max() - data[column].min())
display(data)
data.plot.bar()
```

|   | Возраст | Доход  |   | Возраст | Доход    |
|---|---------|--------|---|---------|----------|
| 0 | 19      | 15000  | 0 | 0.04    | 0.060914 |
| 1 | 67      | 120000 | 1 | 1.00    | 0.593909 |
| 2 | 45      | 200000 | 2 | 0.56    | 1.000000 |
| 3 | 28      | 150000 | 3 | 0.22    | 0.746193 |
| 4 | 17      | 3000   | 4 | 0.00    | 0.000000 |

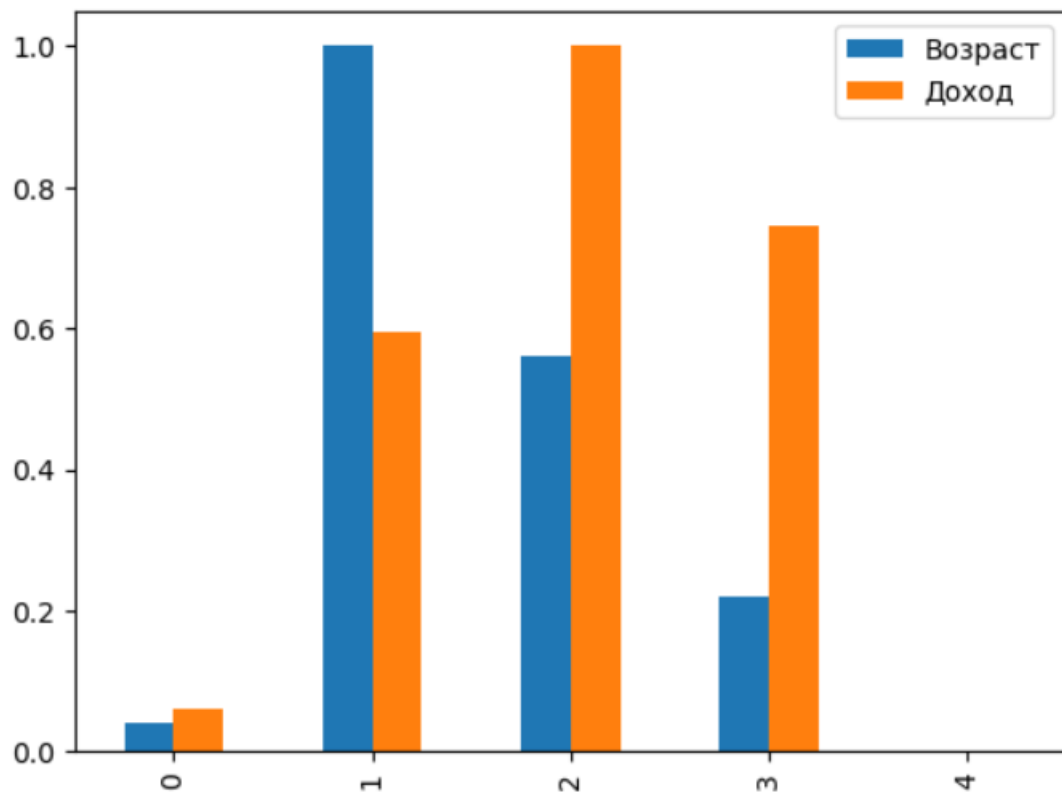


Рис.9. Нормализация данных

## Стандартизация

Во многих сценариях для стабилизации дисперсии и минимизации асимметрии желательно привести распределение данных к распределению, близкому к **гауссовскому**.

**Стандартизация** преобразует данные так, чтобы среднее значение стало 0, а стандартное отклонение 1. Стандартизация масштабирует значения с учетом стандартного отклонения, что снижает влияние выбросов. На рис.10 стандартизация столбцов набора выполнена различными способами.

```
import pandas as pd
import numpy as np
#Scipy – библиотека научных расчетов
from scipy import stats
data=pd.read_excel(r"Нормализация.xlsx")
print(data)
#Стандартизация с помощью встроенных функций pandas mean() и std(), которые будут давать средние и стандартные отклонения данных столбца
data['Возраст']=(data['Возраст']-data['Возраст'].mean())/data['Возраст'].std()
#Стандартизация с помощью библиотеки scipy позволяет преобразовать данные к виду, где график их частоты будет иметь колоколообразную форму
data['Доход'] = stats.zscore(data['Доход'])
print(data)
import matplotlib.pyplot as plt
data['Возраст'].plot(y="Возраст", kind="bar", title="Стандартизация столбцов", legend=True,color='red')
data['Доход'].plot(y="Доход", kind="line", title="Стандартизация столбцов", legend=True,color='green')
```

|   | Возраст   | Доход     |
|---|-----------|-----------|
| 0 | 19        | 15000     |
| 1 | 67        | 120000    |
| 2 | 45        | 200000    |
| 3 | 28        | 150000    |
| 4 | 17        | 3000      |
|   | Возраст   | Доход     |
| 0 | -0.773889 | -1.075266 |
| 1 | 1.519116  | 0.291597  |
| 2 | 0.468155  | 1.333017  |
| 3 | -0.343951 | 0.682130  |
| 4 | -0.869431 | -1.231479 |

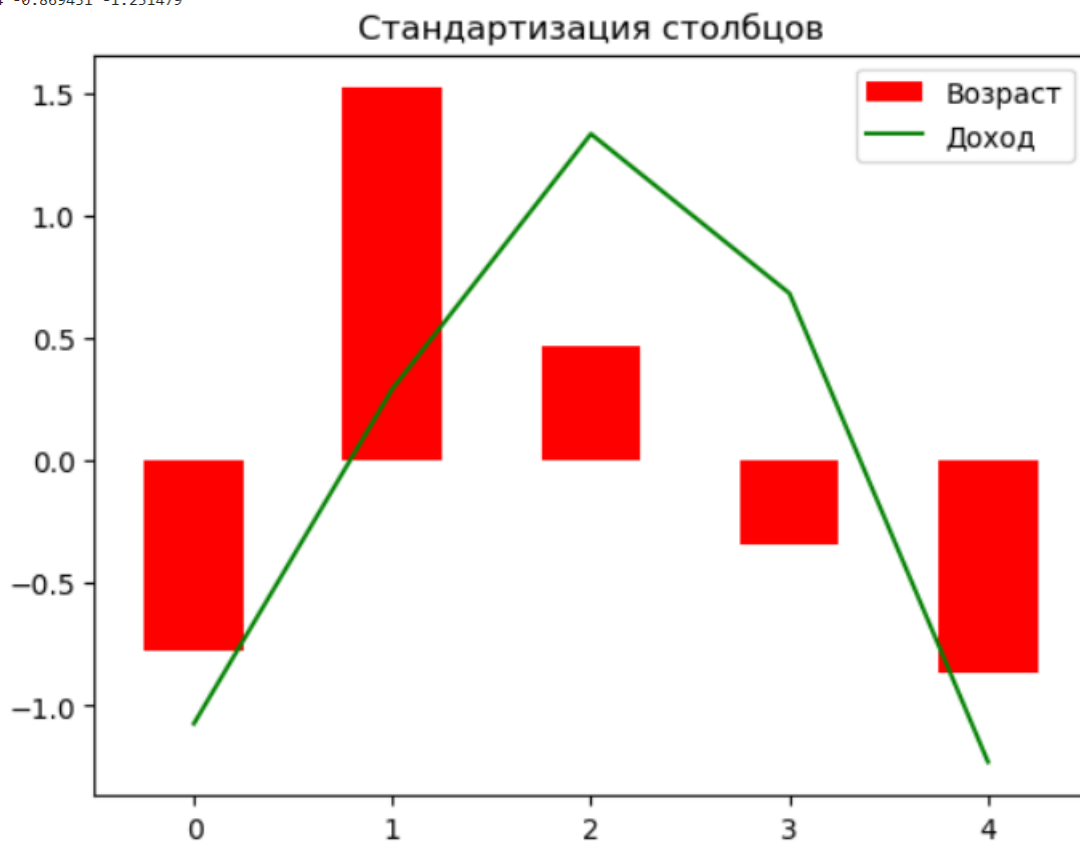


Рис.10. Стандартизация данных



Основное отличие между нормализацией и стандартизацией заключается в том, что нормализация приводит данные к определенному диапазону, а стандартизация — к определенному распределению данных.

Нормализация и стандартизация в Logiном не представлены отдельными компонентами, но доступны как отдельные настройки в мастерах следующих узлов:

- Линейная регрессия;
- Логистическая регрессия;
- Нейросеть (классификация);
- Нейросеть (регрессия);
- Кластеризация;
- EM Кластеризация;
- ARIMAX;
- Самоорганизующиеся сети.

## Отбор признаков

Результатом отбора признаков являются дополнительные сгенерированные характеристики, а также обработанные исходные данные.

**Отбор признаков** (feature selection) — это оценка влияния каждого признака на целевой столбец набора и отсеечение ненужных.

Существует большое количество алгоритмов, позволяющих выполнить преобразование набора со слишком большим количеством признаков в управляемое подмножество.

Как и в случае с построением признаков, для разных типов данных оптимальны разные способы. Более того, при выборе алгоритма необходимо учитывать цели исследования обрабатываемого датасета.

В рамках процедуры анализа данных и отбора признаков нередко вычисляется корреляция между признаками. Корреляция используется для отбора признаков в машинном обучении, так как «хорошие» переменные сильно коррелируют с целевой переменной. При этом переменные должны коррелировать с целевым показателем, но не должны коррелировать между собой.

## Коэффициент корреляции

**Корреляция** — это мера линейной связи двух или более переменных. При помощи корреляции можно предсказать одну переменную через другую. Логика использования

этого метода для выбора характеристик заключается в том, что «хорошие» переменные сильно коррелируют с целевой переменной.

На рис.11 приведена функция для вычисления корреляции переменных. На вход подается набор данных и порог (учитываются и положительное и отрицательное значения, поэтому указывайте только положительное). Функция возвращает коррелирующие столбцы и их коэффициент.

```
import pandas as pd
import numpy as np
data=pd.read_csv(r"Клиенты.txt",encoding='utf-8',sep='\t')
def correl(X_train, limit):
    # Матрица корреляции method='spearman' method='kendall'
    cor = X_train.corr(method='pearson')
    return cor[abs(cor) > limit]
    print(cor)
correl(data[['ВозрастнаяГруппа', 'КоличествоПокупок']], 0.2)
```

|                   | ВозрастнаяГруппа | КоличествоПокупок |
|-------------------|------------------|-------------------|
| ВозрастнаяГруппа  | 1.000000         | -0.320209         |
| КоличествоПокупок | -0.320209        | 1.000000          |

Рис.11. Вычисление корреляции переменных набора

Для выполнения корреляционного анализа в Logitom настройки **компонента Корреляционный анализ** включают список флагов для выбора алгоритма корреляции — рис.12.

При расчете коэффициента корреляции **Пирсона** цифровые данные считаются "непрерывными" вне зависимости непрерывные они или дискретные. Если на вход подаются текстовые данные (дискретные) как на рис.5, то расчёт корреляции по **Пирсону** происходит следующим образом:

- данные сортируются по алфавиту (по возрастанию),
- каждой уникальной записи присваивается порядковый номер,
- рассчитывается корреляция по Пирсону для полученных уникальных номеров.

## Корреляционный анализ

- ☒ Коэффициент корреляции Пирсона
 ☐ Экстремум взаимнокорреляционной функции  
☐ Коэффициент Тау-б Кендалла
 ☐ Коэффициент корреляции Спирмена

| Входные колонки                                                                    |  | Набор 1                             | Набор 2                             |
|------------------------------------------------------------------------------------|--|-------------------------------------|-------------------------------------|
| 12 № п/п                                                                           |  | <input type="checkbox"/>            | <input type="checkbox"/>            |
| ab Регион                                                                          |  | <input checked="" type="checkbox"/> | <input type="checkbox"/>            |
| 9.0 Объем услуг связи, оказанных населению, на одного жителя, руб. (y)             |  | <input type="checkbox"/>            | <input checked="" type="checkbox"/> |
| 9.0 Наличие квартирных телефонных аппаратов сети общего пользования на 1000 чел... |  | <input type="checkbox"/>            | <input type="checkbox"/>            |
| 9.0 Число подключенных абонентских устройств подвижной радиотелефонной связи ...   |  | <input type="checkbox"/>            | <input type="checkbox"/>            |
| 9.0 Число активных абонентов фиксированного широкополосного доступа к сети Инте... |  | <input type="checkbox"/>            | <input type="checkbox"/>            |
| 9.0 Число активных абонентов подвижной радиотелефонной связи, использующих ши...   |  | <input type="checkbox"/>            | <input type="checkbox"/>            |

| Выходной набор данных |              |                |      |                                                                 |
|-----------------------|--------------|----------------|------|-----------------------------------------------------------------|
| #                     | ab Поле1.Имя | ab Поле1.Метка | ab Г | ab Поле2.Метка                                                  |
| 1                     | Region       | Регион         | О... | Объем услуг связи, оказанных населению, на одного жителя, ру... |

9.0 Пирсона -0,23

Рис.12. Настройки компонента Корреляционный анализ в Loginom

## Задание для самостоятельной работы

Для построения и отбора признаков с применением встроенных компонентов Loginom и узла Python используйте наборы данных (datasets), находящиеся в открытом доступе. Например, платформа **kaggle.com** (рис.13) дает свободный доступ зарегистрированным пользователям к коллекции аннотированных наборов данных. Зарегистрируйтесь и скачайте наборы данных для исследования [2].

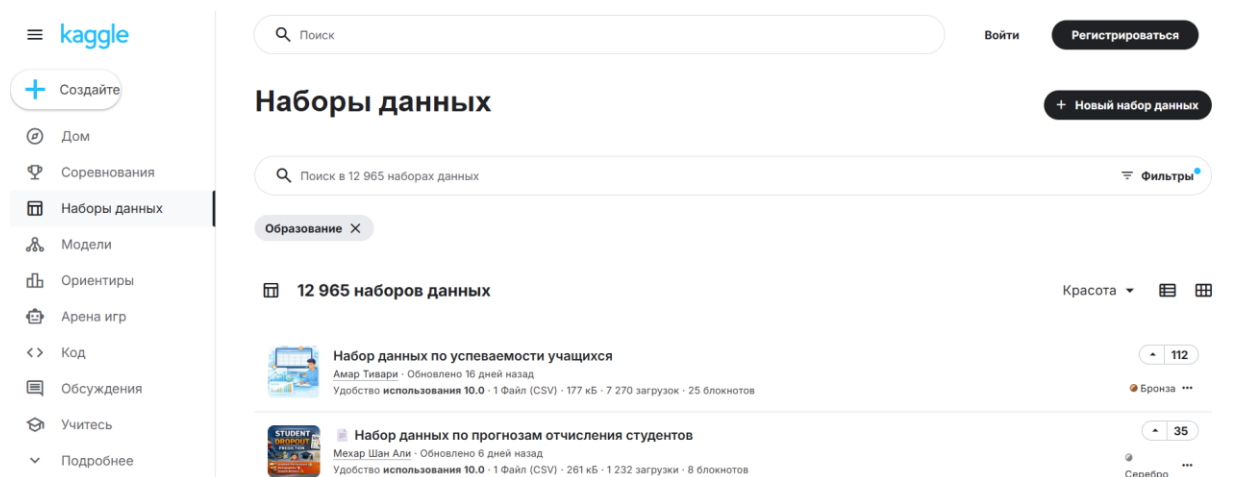


Рис. 13. Наборы данных, размещенные на платформе kaggle.com

Реализуйте рассмотренные в лабораторной работе этапы построения и отбора признаков для наборов данных как с использованием Python в соответствующем узле платформы, так и с помощью визуальных инструментов этой платформы.

Сформулируйте аргументированные заключения о преимуществах или недостатках различных методов анализа.

#### Источники

1. Программирование в Loginom: Python vs JavaScript. URL: <https://loginom.ru/blog/use-javascript-python?ysclid=m4a4miot72967154652> (дата просмотра 28.02.2026)
2. Kaggle. URL: <https://www.kaggle.com/datasets>. (дата просмотра 28.02.2026)